1. Introduction

What is xue-python?

xue-python is an enhanced Python distribution built on CPython 3.12 and 3.14 (free-threaded), developed by **OpenCan.ai**. It adds a standard library of modules for scientific computing, AI/ML, robotics, and safety-critical programming that Python lacks out of the box.

The name *xue* means "learn" in Chinese. Every module is designed for production use in research and engineering workflows.

Result & Option Types

Rust-inspired explicit error handling. No more silent None failures.

Physical Units

SI unit system with dimension checking. Catches meters + seconds errors at runtime.

Automatic Differentiation

Reverse-mode AD with computation graph. Gradients, Jacobians, and higher-order derivatives.

Tensor with SIMD + GPU

Shape-checked tensors with AVX2 vectorization and optional CUDA GPU acceleration.

Design by Contract

@requires, @ensures, @invariant decorators for preconditions and postconditions.

Secret Type

Prevents accidental logging of API keys, tokens, and passwords.

Multiple Dispatch

Julia-style dispatch on all argument types, not just self.

Sandboxed Imports

Capability-based import restrictions for untrusted code.

LLM Diagnostics

Optional AI-powered error explanations and natural language contract translation.

Architecture

xue-python is a standard CPython fork. All existing Python packages work unchanged. The `xue` package is a standard library module (like `json` or `math`). Performance-critical modules have C extension backends that are loaded automatically when available.

Component	Description
Base	CPython 3.12.13 (stable) and 3.14.3 (free-threaded)
Library	`Lib/xue/` — 11 Python modules
C Extensions	`_units_accel.c`, `_autodiff_accel.c`, `_tensor_accel.c`
CUDA Kernels	`_xue_cuda_kernels.cu` (optional, runtime-loaded)
License	MIT (xue extensions) + PSF (CPython base)

2. License

- **xue modules** (`Lib/xue/`, C extensions, CUDA kernels) — MIT License.
- **CPython base** — PSF License Agreement.

You may use xue-python in commercial and open-source projects without restriction. See `LICENSE.xue` for the full MIT text.

3. Installation

From Pre-built Distribution

```
# Python 3.12 (stable)
tar xzf xue-python-3.12.13-linux-x86_64.tar.gz
export PATH="/opt/xue-python/bin:$PATH"
python3 -c "import xue; print(xue.__version__)"

# Python 3.14t (free-threaded, no GIL)
tar xzf xue-python-3.14.3t-linux-x86_64.tar.gz
export PATH="/opt/xue-python-314t/bin:$PATH"
python3 -c "import xue; print(xue.__version__)"
```

Building from Source

```
# Prerequisites (Ubuntu/Debian)
sudo apt install build-essential libssl-dev libffi-dev zlib1g-dev
# Optional: CUDA toolkit for GPU support
sudo apt install nvidia-cuda-toolkit

# Build Python 3.12 fork
cd xue-python
./configure --prefix=/opt/xue-python --enable-optimizations
make -j$(nproc)
make install

# Build C extensions
cd ../xue-native
bash build.sh
```

System Requirements

Requirement	Minimum	Recommended
OS	Linux x86_64	Ubuntu 22.04+
GCC	11+	13+ (for AVX2)
CPU	x86_64 with SSE4	AVX2 capable (Haswell+)
GPU (optional)	CUDA 11.0+	CUDA 12.0+
RAM	512 MB	2 GB+

4. Quick Start

```
from xue.result import Ok, Err
from xue.units import meters, seconds, kg
from xue.autodiff import Variable, grad
from xue.tensor import Tensor, float32

# Physical units with dimension checking
distance = 100 * meters
time = 9.58 * seconds
speed = distance / time           # Quantity(10.44..., m/s)
# distance + time                 # DimensionError!

# Automatic differentiation
x = Variable(3.0)
y = x ** 2 + 2 * x + 1
y.backward()
print(f"dy/dx at x=3: {x.grad}")  # 8.0

# Functional gradient
df = grad(lambda x: x ** 3 + 2 * x)
print(f"f'(3) = {df(3.0)}")       # 29.0

# Tensors with shape checking and SIMD
a = Tensor[[3, 4], float32]([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
b = Tensor[[4, 2], float32]([[1,2], [3,4], [5,6], [7,8]])
c = a @ b                         # Shape-checked: (3,4) @ (4,2) = (3,2)
print(c.shape)                    # (3, 2)

# Result type for explicit error handling
def divide(a, b):
    if b == 0:
        return Err("division by zero")
    return Ok(a / b)

result = divide(10, 3).map(lambda x: round(x, 2))
print(result.unwrap())            # 3.33
```

5. Result & Option Types

Module: `xue.result`

Provides `Result[T, E]` and `Option[T]` types inspired by Rust. Makes error paths visible in type signatures instead of using exceptions or None.

Result[T, E]

```python
from xue.result import Ok, Err, Result

def parse_int(s: str) -> Result[int, str]:
    try:
        return Ok(int(s))
    except ValueError:
        return Err(f"cannot parse '{s}' as int")

# Pattern matching (Python 3.10+)
match parse_int("42"):
    case Ok(value):
        print(f"Got: {value}")
    case Err(error):
        print(f"Error: {error}")

# Method chaining
result = (parse_int("10")
          .map(lambda x: x * 2)
          .and_then(lambda x: Ok(x + 1) if x < 100 else Err("too large"))
          .unwrap_or(0))
```

Option[T]

```python
from xue.result import Some, Nothing, Option

def find_user(id: int) -> Option[dict]:
    users = {1: {"name": "Alice"}, 2: {"name": "Bob"}}
    if id in users:
        return Some(users[id])
    return Nothing()

name = find_user(1).map(lambda u: u["name"]).unwrap_or("Unknown")
```

Method	Description
`.is_ok()` / `.is_err()`	Check variant
`.unwrap()`	Get value or raise
`.unwrap_or(default)`	Get value or return default
`.map(fn)`	Transform the Ok/Some value
`.and_then(fn)`	Chain operations that return Result/Option
`.or_else(fn)`	Handle the error case

6. Secret Type

Module: `xue.secret`

Wraps sensitive data (API keys, tokens, passwords) and prevents accidental exposure through repr, str, logging, or JSON serialization.

```
from xue.secret import Secret

api_key = Secret("sk-abc123def456")

print(api_key)              # Secret(****)
f"{api_key}"                # Secret(****)
repr(api_key)               # Secret(****)

# Intentional access
token = api_key.expose()    # "sk-abc123def456"

# Constant-time comparison (prevents timing attacks)
if api_key == Secret("sk-abc123def456"):
    print("Authenticated")

# Type-safe function signatures
def connect(token: Secret[str]) -> None:
    requests.get(url, headers={"Authorization": token.expose()})
```

Security note: Set `XUE_SCRUB_SECRETS=1` to also redact Secret values from tracebacks.

7. Design by Contract

Module: `xue.contracts`

Adds `@requires` (preconditions), `@ensures` (postconditions), and `@invariant` (class invariants) decorators.

```
from xue.contracts import requires, ensures, invariant

@requires(lambda a, b: b != 0, "divisor must not be zero")
@ensures(lambda result, a, b: abs(result * b - a) < 1e-10, "result * b must equal a")
def divide(a: float, b: float) -> float:
    return a / b

divide(10, 3)   # OK
divide(10, 0)   # ContractError: divisor must not be zero

@invariant(lambda self: self.balance >= 0, "balance must be non-negative")
class BankAccount:
    def __init__(self, balance: float):
        self.balance = balance

    @requires(lambda self, amount: amount > 0, "amount must be positive")
    def withdraw(self, amount: float):
        self.balance -= amount
```

Tip: Disable contract checking in production with `XUE_CONTRACTS=0` or `xue.contracts.set_enabled(False)` for zero overhead.

8. Strict Typing

Module: `xue.strict`

Runtime type checking based on function annotations. Catches type mismatches at function boundaries instead of letting them propagate.

```
from xue.strict import checked

@checked
def compute(x: float, y: float) -> float:
    return x + y

compute(1.0, 2.0)      # OK -> 3.0
compute("a", "b")      # TypeError: x: expected float, got str

# Enable for entire module
from xue.strict import strict_module
strict_module(__name__)
```

Enable globally with `XUE_STRICT=1` or `python --strict`.

9. Physical Units

Module: `xue.units`

A complete SI unit system with dimension analysis. Prevents unit mismatch errors in scientific and robotics code. Backed by a C extension for high-performance arithmetic.

Basic Usage

```
from xue.units import meters, seconds, kg, kilometers, newtons, degrees, radians

distance = 100 * meters
time = 9.58 * seconds
speed = distance / time              # Quantity(10.44..., m/s)

mass = 75 * kg
force = mass * (speed / time)        # Quantity(..., kg*m/s^2)

# Dimension mismatch caught at runtime
distance + time                      # DimensionError!

# Unit conversion
distance_km = distance.to(kilometers)  # 0.1

# Angle conversion
angle = 180 * degrees
angle.to(radians)                    # 3.14159...
```

Available Units

Category	Units
Length	`meters`, `kilometers`, `centimeters`, `millimeters`
Mass	`kilograms` (`kg`), `grams`
Time	`seconds` (`s`), `milliseconds`, `microseconds`, `nanoseconds`, `minutes`, `hours`
Electrical	`amperes`, `volts`, `watts`
Derived	`newtons`, `pascals`, `joules`, `hertz`, `newton_meters`
Angle	`radians`, `degrees`, `rpm`
Other	`kelvin`, `moles`, `candelas`

Custom Units

```
from xue.units import Dimension, Unit

# Define a custom dimension
pressure = Dimension((-1, 1, -2, 0, 0, 0, 0))  # L^-1 * M * T^-2
bar = Unit(pressure, 1e5, "bar", "bar")
atm = Unit(pressure, 101325.0, "atmosphere", "atm")

p = 2.5 * bar
p.to(atm)  # 2.467...
```

> **Performance:** The C extension packs 7 dimension exponents into a single 64-bit integer for single-instruction comparison. Quantity arithmetic is **9–15x faster** than pure Python.

10. Automatic Differentiation

Module: `xue.autodiff`

Reverse-mode automatic differentiation via operator overloading. Computes exact gradients, Jacobians, and higher-order derivatives. No symbolic math or finite differences.

Variable and Backward

```
from xue.autodiff import Variable, grad, value_and_grad, jacobian

# Build computation graph, then differentiate
x = Variable(3.0, name="x")
y = Variable(2.0, name="y")
z = x ** 2 + 2 * x * y + y ** 2   # (x + y)^2

z.backward()
print(x.grad)   # 10.0  (2x + 2y)
print(y.grad)   # 10.0  (2x + 2y)
```

Functional API

```
# grad() returns a function
df = grad(lambda x: x ** 3 + 2 * x)
df(3.0)      # 29.0   (3x^2 + 2)
df(1.0)      # 5.0

# value_and_grad() returns both
vg = value_and_grad(lambda x: (x - 3) ** 2)
val, g = vg(5.0)   # val=4.0, g=4.0

# Jacobian matrix
def f(x):
    return [x[0] + x[1], x[0] * x[1]]
J = jacobian(f, [2.0, 3.0])
# [[1.0, 1.0], [3.0, 2.0]]
```

Supported Operations

Category	Operations
Arithmetic	`+`, `-`, `*`, `/`, `**`, unary `-`
Transcendental	`.exp()`, `.log()`, `.sqrt()`
Trigonometric	`.sin()`, `.cos()`, `.tan()`, `.tanh()`
Activation	`.sigmoid()`, `.relu()`
Other	`.abs()`

> **Performance:** The C extension replaces Python closures with an enum-based operation table and C-level topological sort. Backward pass uses switch-dispatch instead of function calls. **9–11x faster** than pure Python.

11. Tensor

Module: `xue.tensor`

N-dimensional tensor type with runtime shape validation, SIMD-accelerated operations, and optional CUDA GPU support.

Creating Tensors

```
from xue.tensor import Tensor, float32, float64, zeros, ones, eye, arange

# From nested lists (shape inferred)
a = Tensor([[1, 2, 3], [4, 5, 6]])        # shape (2, 3), dtype float64

# With explicit shape and dtype
b = Tensor[[3, 4], float32]([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Factory functions
z = zeros((3, 3))
o = ones((2, 4), dtype=float32)
I = eye(3)
r = arange(0, 10, 0.5)
```

Arithmetic (SIMD-accelerated)

```
a = Tensor([[1, 2], [3, 4]])
b = Tensor([[5, 6], [7, 8]])

c = a + b        # elementwise add (AVX2: 4 doubles/cycle)
d = a * b        # elementwise multiply
e = a - b        # elementwise subtract
f = a / b        # elementwise divide
g = a * 2.5      # scalar multiply
h = -a           # negate
```

Matrix Multiply (SIMD tiled)

```
a = Tensor([[1, 2, 3], [4, 5, 6]])    # (2, 3)
b = Tensor([[1, 2], [3, 4], [5, 6]])  # (3, 2)
c = a @ b                             # (2, 2) - shape checked!

# Shape mismatch caught:
d = Tensor([[1, 2, 3]])               # (1, 3)
d @ b                                 # OK: (1,3) @ (3,2) = (1,2)
d @ a                                 # ShapeError: inner dimensions 3 != 2
```

Shape Operations

```
t = Tensor([[1, 2, 3, 4], [5, 6, 7, 8]])   # (2, 4)
t.reshape(4, 2)       # (4, 2)
t.T                   # transpose: (4, 2)
t.sum()               # 36.0
t.mean()              # 4.5
t.max()               # 8.0
t.min()               # 1.0
t.tolist()            # [[1.0, 2.0, ...], ...]
```

GPU Support (CUDA)

```
# Transfer to GPU (requires NVIDIA GPU + CUDA)
gpu_tensor = tensor.to_gpu()

# Operations on GPU tensors use CUDA kernels
c = gpu_a + gpu_b       # CUDA elementwise add
d = gpu_a @ gpu_b       # CUDA tiled matmul

# Transfer back to CPU
cpu_tensor = gpu_tensor.to_cpu()
```

> **Note:** GPU support requires an NVIDIA GPU with CUDA 11.0+. If no GPU is detected, `.to_gpu()` raises `RuntimeError`. All operations have CPU fallbacks.

12. Multiple Dispatch

Module: `xue.dispatch`

Julia-style multiple dispatch based on all argument types.

```
from xue.dispatch import multimethod

@multimethod
def add(a: int, b: int) -> int:
    return a + b

@multimethod
def add(a: float, b: float) -> float:
    return a + b

@multimethod
def add(a: list, b: list) -> list:
    return a + b

add(1, 2)        # 3      (int, int)
add(1.5, 2.5)    # 4.0    (float, float)
add([1], [2])    # [1, 2] (list, list)
```

13. Unicode Operators

Module: `xue.unicode_ops`

Registers a custom codec that translates Unicode mathematical operators to Python equivalents during source parsing.

```
# coding: xue-unicode

if x ≤ 10 ∧ y ≠ 0:
    result = x ÷ y
```

Unicode	Python	Unicode	Python
`≤`	`<=`	`≥`	`>=`
`≠`	`!=`	`≡`	`==`
`×`	`*`	`÷`	`/`
`¬`	`not`	`∧`	`and`
`∨`	`or`	`←`	`=`
`∈`	`in`	`∉`	`not in`
`√`	`math.sqrt`	`∞`	`float('inf')`
`π`	`3.14159...`	`τ`	`6.28318...`

Activate globally: `import xue.unicode_ops; xue.unicode_ops.register()`

14. Sandboxed Imports

Module: `xue.sandbox`

Capability-based import restrictions for untrusted code. Prevents imported modules from accessing network, filesystem, subprocess, or ctypes.

```
from xue.sandbox import sandboxed_import, SandboxPolicy

policy = SandboxPolicy(
    allow_network=False,
    allow_filesystem=False,
    allow_subprocess=False,
    allow_ctypes=False,
)
untrusted = sandboxed_import("untrusted_plugin", policy=policy)
# Any network/file access by the plugin raises SandboxViolation

# Context manager form
from xue.sandbox import sandbox
with sandbox(allow_network=False, allow_subprocess=False):
    import some_plugin
```

15. LLM Integration

Module: `xue.llmhook`

Optional AI-powered diagnostics. Zero overhead when disabled.

```
from xue.llmhook import explain, ask, configure

# Configure backend (Ollama, OpenAI, or Unix socket)
configure(backend="http", url="http://localhost:11434/api/generate", model="llama3")

# Explain an error
try:
    result = compute(data)
except Exception as e:
    explanation = explain(e)
    print(explanation)  # Human-readable fix suggestion

# Ask a question about your code
answer = ask("Why is this function returning None?")
```

Activate with: `python --llm` or `XUE_LLM_HOOK=1`

16. Native C Extensions

Three performance-critical modules have C extension backends that load automatically. Pure Python fallbacks are always available.

Module	C Extension	Key Optimizations
`xue.units`	`_units_accel.c`	Dimension exponents packed into int64; cached Unit objects; Quantity arithmetic in C
`xue.autodiff`	`_autodiff_accel.c`	Enum-based op table instead of closures; C topological sort; switch-dispatch backward
`xue.tensor`	`_tensor_accel.c`	Raw `double*` storage; AVX2 SIMD; tiled matmul; optional CUDA

> **Fallback:** If C extensions are not compiled, the Python implementations are used automatically. The API is identical — no code changes needed.

17. SIMD Vectorization

The tensor C extension auto-detects the CPU architecture at compile time and uses the appropriate SIMD instruction set:

Platform	SIMD	Doubles/Cycle	Register Width
x86_64 (Intel/AMD)	AVX2	4	256-bit
aarch64 (Apple Silicon, ARM servers)	NEON	2	128-bit
Other	Scalar fallback	1	N/A

Vectorized Operations

- **Elementwise:** add, subtract, multiply, divide, negate, scalar multiply, scalar add
- **Reduction:** sum (horizontal SIMD sum)
- **Matrix multiply:** Tiled algorithm (32x32 tiles) with SIMD inner loop

All operations release the GIL during computation via `Py_BEGIN_ALLOW_THREADS`, enabling true parallelism on the 3.14t free-threaded build.

18. CUDA GPU Support

GPU acceleration is provided by a separate CUDA kernel library loaded at runtime via dynamic loading (`dlopen` on Linux/macOS, `LoadLibrary` on Windows). No compile-time CUDA dependency for the main extension.

CUDA Kernels

Kernel	Description
`cuda_elementwise_add/sub/mul/div`	Parallel elementwise operations (256 threads/block)
`cuda_scalar_mul`	Parallel scalar multiplication
`cuda_matmul`	Shared-memory tiled matrix multiply (16x16 tiles)

Building CUDA Kernels

```
# Requires nvcc (NVIDIA CUDA Compiler)
nvcc -shared -O3 -Xcompiler -fPIC \
    _xue_cuda_kernels.cu -o _xue_cuda_kernels.so -lcudart

# Copy to xue library directory
cp _xue_cuda_kernels.so /path/to/lib/pythonX.Y/xue/
```

19. Benchmarks

Measured on x86_64 (GCC 13.3, AVX2), comparing pure Python vs C extensions.

Units (Quantity arithmetic)

Operation	Python	C Extension	Speedup
Quantity create	2.7M ops/s	11.6M ops/s	4.3x
Quantity add	1.9M ops/s	17.0M ops/s	8.9x
Quantity multiply	448K ops/s	6.6M ops/s	14.8x
Quantity divide	427K ops/s	6.5M ops/s	15.2x
Unit conversion	3.5M ops/s	10.3M ops/s	2.9x

Autodiff (Variable + backward)

Expression	Python	C Extension	Speedup
x^2+2x+1 + backward	141K ops/s	1.23M ops/s	8.7x
MLP-like + backward	79K ops/s	728K ops/s	9.2x

Tensor (SIMD + tiled matmul)

Operation	Python	C Extension	Speedup
Elementwise add (100x100)	802 ops/s	251K ops/s	313x
Elementwise mul (100x100)	840 ops/s	253K ops/s	301x
Matmul (100x100)	8 ops/s	2,870 ops/s	340x
Matmul (256x256)	0.5 ops/s	234 ops/s	433x
Sum reduction (100x100)	8.9K ops/s	435K ops/s	49x

20. Building from Source

Directory Structure

```
xue/                          # Git repository (~630 KB)
  xue-modules/                # Python module source files
    __init__.py
    result.py                  # Result[T,E] and Option[T]
    secret.py                  # Secret type
    contracts.py               # @requires, @ensures, @invariant
    strict.py                  # Runtime type checking
    units.py                   # Physical units (+ C accel)
    autodiff.py                # Automatic differentiation (+ C accel)
    tensor.py                  # Tensor type (+ C accel)
    dispatch.py                # Multiple dispatch
    unicode_ops.py             # Unicode operator aliases
    sandbox.py                 # Sandboxed imports
    llmhook.py                 # LLM integration
    tests/                     # 197 unit tests
  xue-native/                  # C extension source code
    _units_accel.c             # Units C extension
    _autodiff_accel.c          # Autodiff C extension
    _tensor_accel.c            # Tensor C extension (AVX2 + NEON + CUDA)
    _xue_cuda_kernels.cu       # CUDA GPU kernels
    build.sh                   # Cross-platform build script
    benchmark.py               # Performance benchmark
  xue-guide.html               # User guide
  xue-guide.pdf                # User guide (PDF)
  .gitignore

# Hosted separately (www.opencan.ai/sources/xue):
  xue-python-3.12.13.tar.gz  # Full CPython 3.12 fork (89 MB)
```

Platform Support

Platform	SIMD	CUDA	Compiler	Status
Linux x86_64	AVX2	Yes (dlopen)	gcc	Fully tested
macOS x86_64	AVX2	No*	clang	Supported
macOS ARM64 (Apple Silicon)	NEON	No*	clang	Supported
Windows x86_64	AVX2	Yes (LoadLibrary)	MSVC/gcc	Supported

* macOS dropped CUDA support in 10.14; CUDA code compiles but no GPU runtime available.

Build Commands

```
# Build C extensions (auto-detects platform, SIMD, compiler)
cd xue-native
bash build.sh

# The script:
# 1. Detects CPU architecture (x86_64 → AVX2, aarch64 → NEON)
# 2. Detects OS (Linux → -ldl, macOS → libSystem, Windows → kernel32)
# 3. Compiles .c files with -O3 -march=native
# 4. Links against Python includes for 3.12 and 3.14t
# 5. Optionally builds CUDA kernels with nvcc
# 6. Runs verification imports

# Override Python paths if needed:
PYTHON312=/path/to/python3.12 PYTHON314=/path/to/python3.14t bash build.sh
```

21. Testing

```
# Run all tests
python3 -m unittest xue.tests.test_units xue.tests.test_autodiff \
    xue.tests.test_tensor xue.tests.test_result xue.tests.test_secret \
    xue.tests.test_contracts xue.tests.test_dispatch \
    xue.tests.test_strict xue.tests.test_unicode_ops -v

# Run a specific test module
python3 -m unittest xue.tests.test_tensor -v

# Run benchmarks
cd xue-native && python3 benchmark.py
```

> **Test status:** All 197 tests pass on both Python 3.12 and 3.14t (free-threaded). C extensions are API-compatible with pure Python implementations.

22. API Reference

xue.result

Type/Function	Description
`Ok(value)`	Success variant of Result
`Err(error)`	Error variant of Result
`Some(value)`	Present variant of Option
`Nothing()`	Absent variant of Option

xue.units

Type/Function	Description
`Dimension(exponents, name=None)`	Physical dimension (7 SI exponents)
`Unit(dimension, scale, name, symbol)`	Physical unit with scale factor
`Quantity(value, unit)`	Numeric value with unit
`Quantity.to(target_unit)`	Convert to different unit
`DimensionError`	Raised on incompatible dimensions

xue.autodiff

Type/Function	Description
`Variable(data, name="", requires_grad=True)`	Differentiable variable
`Variable.backward()`	Compute gradients via reverse-mode AD
`grad(f, argnum=0)`	Return gradient function
`value_and_grad(f, argnum=0)`	Return (value, gradient) function
`jacobian(f, x)`	Compute Jacobian matrix at point x

xue.tensor

Type/Function	Description
`Tensor(data, shape, dtype)`	N-dimensional tensor
`Tensor[[shape], dtype](data)`	Shape-checked constructor
`zeros(shape, dtype)`	Zero-filled tensor
`ones(shape, dtype)`	One-filled tensor
`eye(n, dtype)`	Identity matrix
`arange(start, stop, step, dtype)`	Range tensor
`Tensor.to_gpu()`	Transfer to CUDA GPU
`Tensor.to_cpu()`	Transfer to CPU
`ShapeError`	Raised on shape mismatch

xue.contracts

Decorator	Description
`@requires(predicate, message)`	Precondition check
`@ensures(predicate, message)`	Postcondition check
`@invariant(predicate, message)`	Class invariant check
`set_enabled(bool)`	Enable/disable contract checking

xue.dispatch

Decorator	Description
`@multimethod`	Register function for multiple dispatch

xue.secret

Type	Description
`Secret(value)`	Wrap sensitive data
`Secret.expose()`	Retrieve the raw value